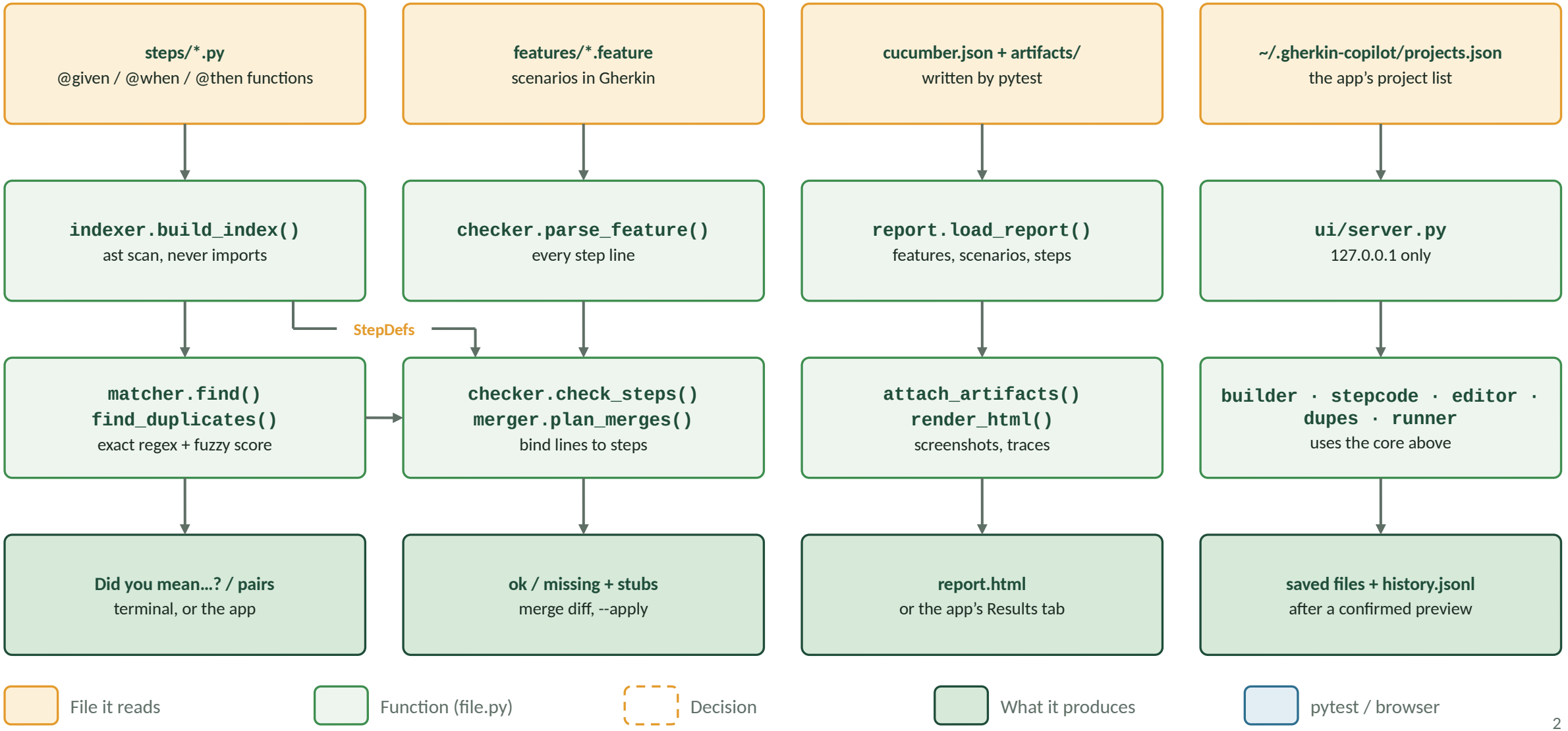
gherkin-copilot: how the data flows

Where every function gets its input, what it does with it, and what it produces. Includes how conftest.py wires the test suite together.

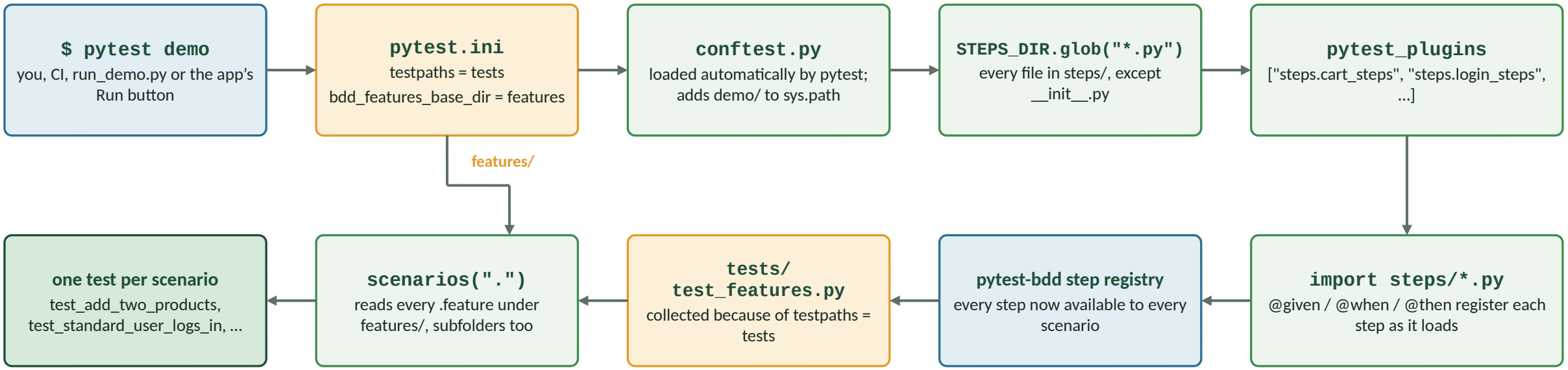
The big picture: four inputs, four pipelines

Every command starts from one of these files. Later slides zoom into each column.



conftest.py, part 1: how pytest finds every step and scenario

What happens when you type pytest, before any test runs.




```python
# demo/conftest.py
STEPS_DIR = Path(__file__).resolve().parent / "steps"
pytest_plugins = [f"steps.{p.stem}" for p in sorted(STEPS_DIR.glob("*.py"))
                  if p.stem != "__init__"]
```

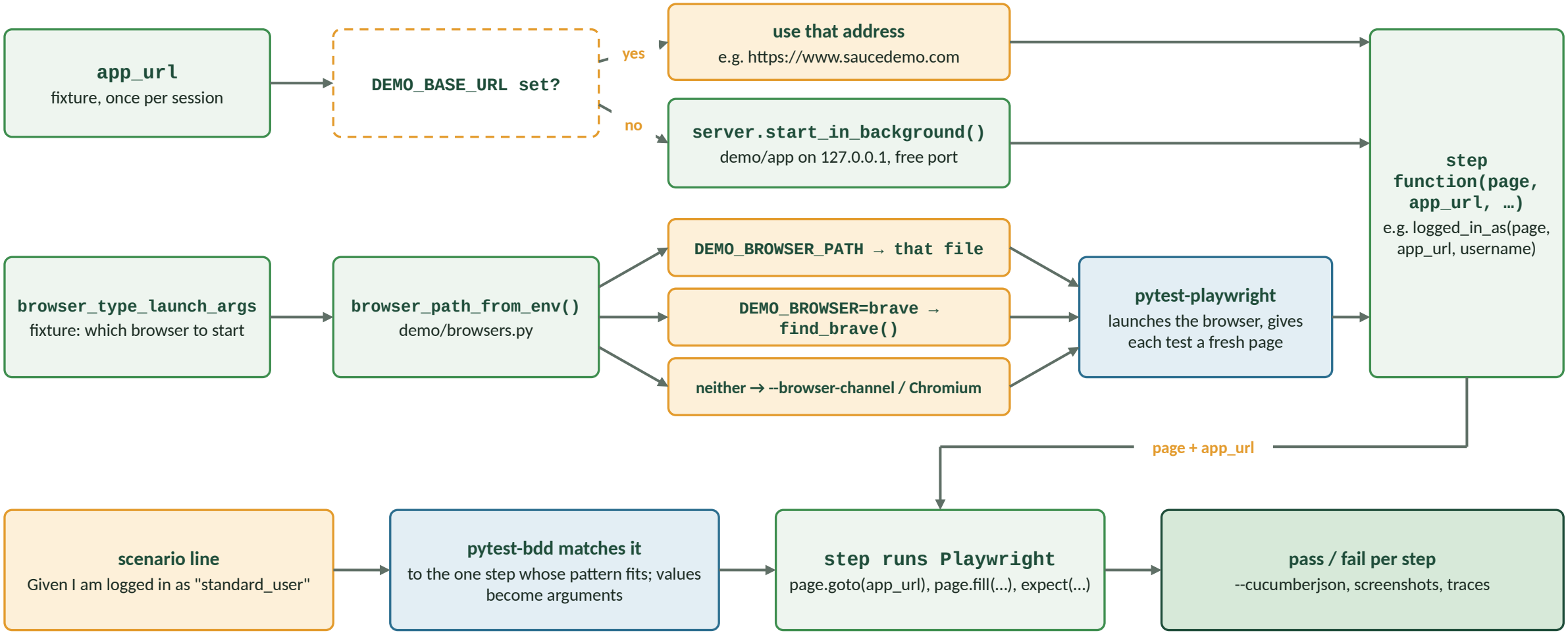
```python
# demo/tests/test_features.py
from pytest_bdd import scenarios
scenarios(".")
```

Why it matters

A new step file (for example one the app writes from action blocks) or a new .feature file needs no registration. Put it in the folder and the next run picks it up.

conftest.py, part 2: what every test gets from its fixtures

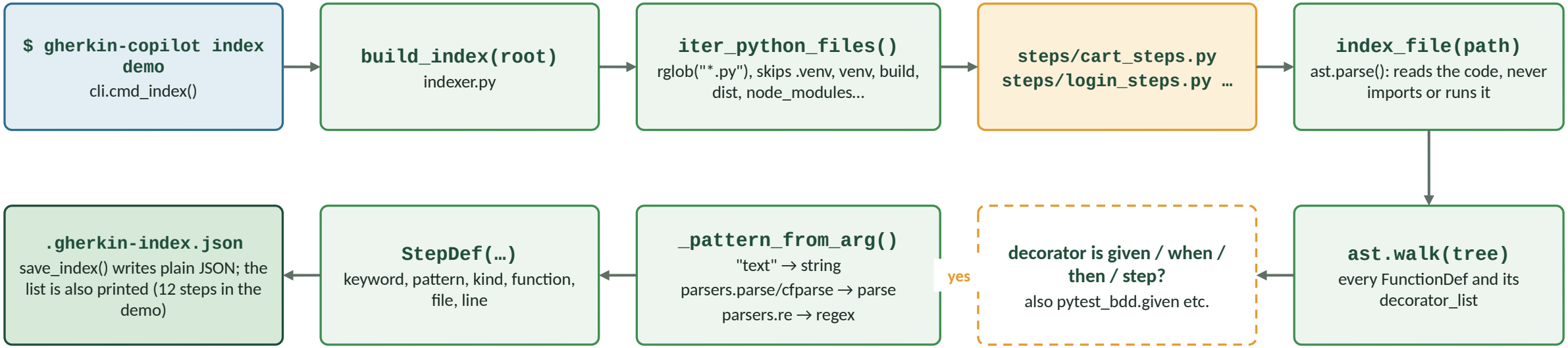
Fixtures are pytest's way of handing ready-made things to a test. conftest.py defines two.



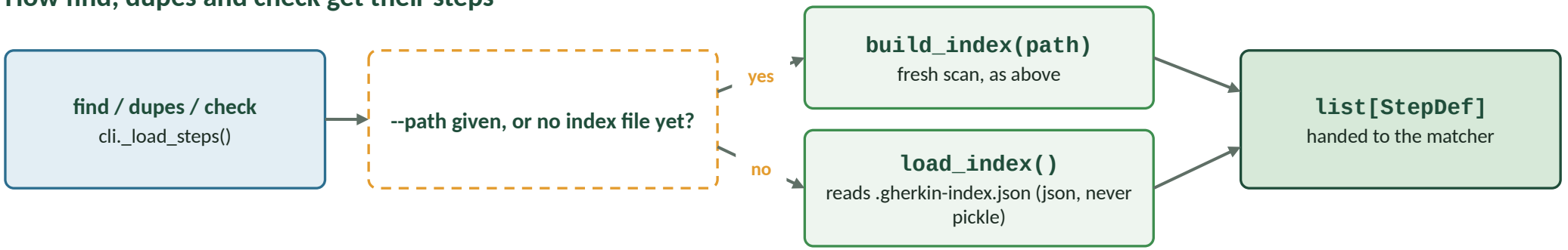
Run afterwards:
the app_url server is shut down when the session ends.

index: where the step list comes from

gherkin-copilot index demo → .gherkin-index.json



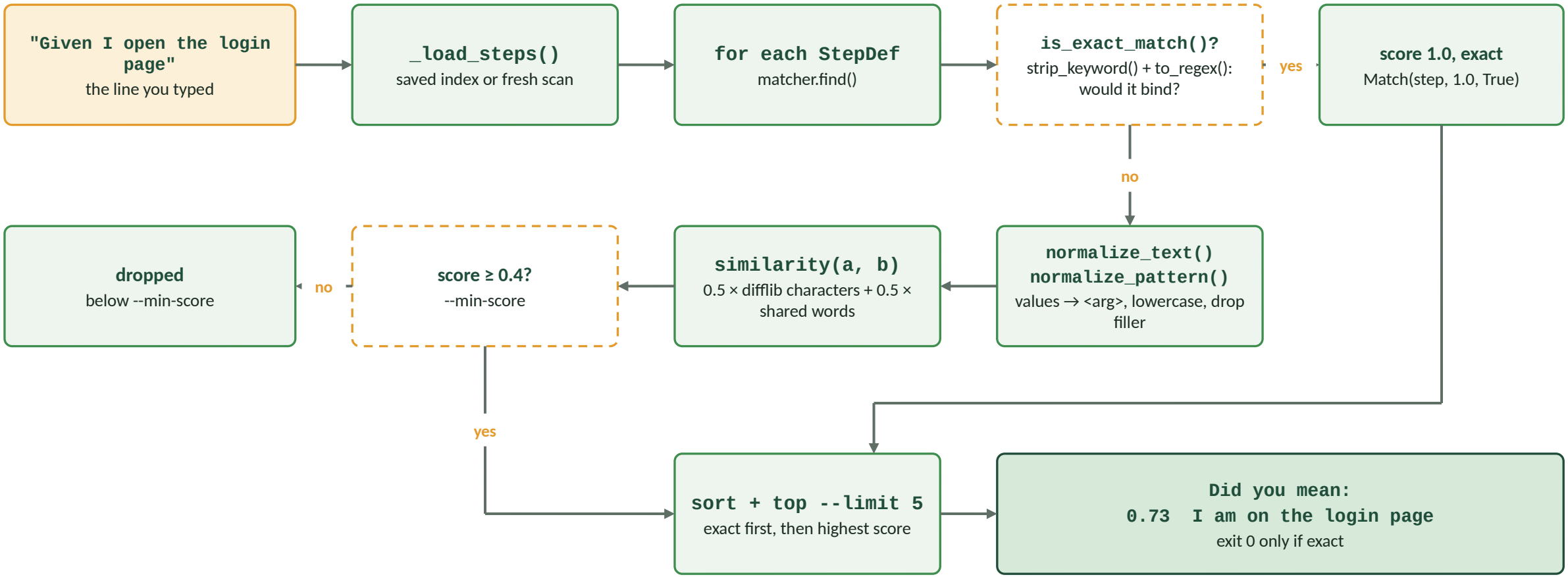
How find, dupes and check get their steps



merge and the app
always scan fresh with build_index(), so they never work from a stale index.

find: exact match first, then “did you mean”

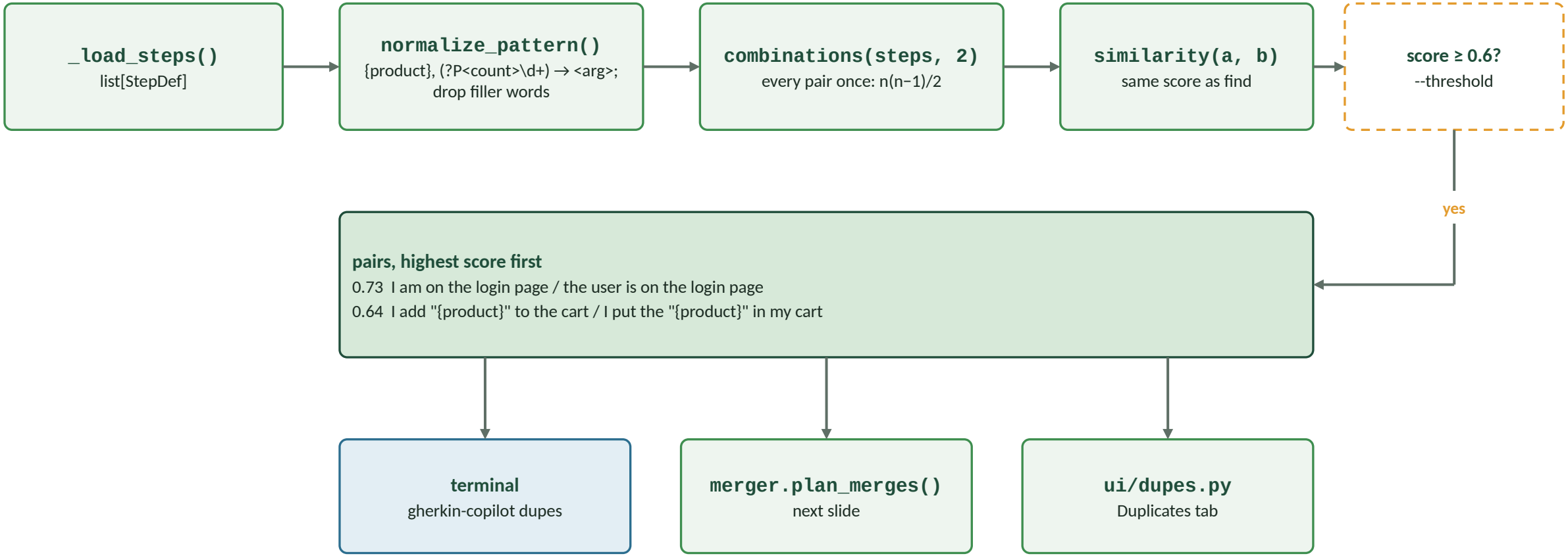
gherkin-copilot find "Given I open the login page"



Worked example:
“open login page” vs “login page”: characters 0.80, words 2 of 3 = 0.67, average 0.73.

dupes: compare every pair of step patterns

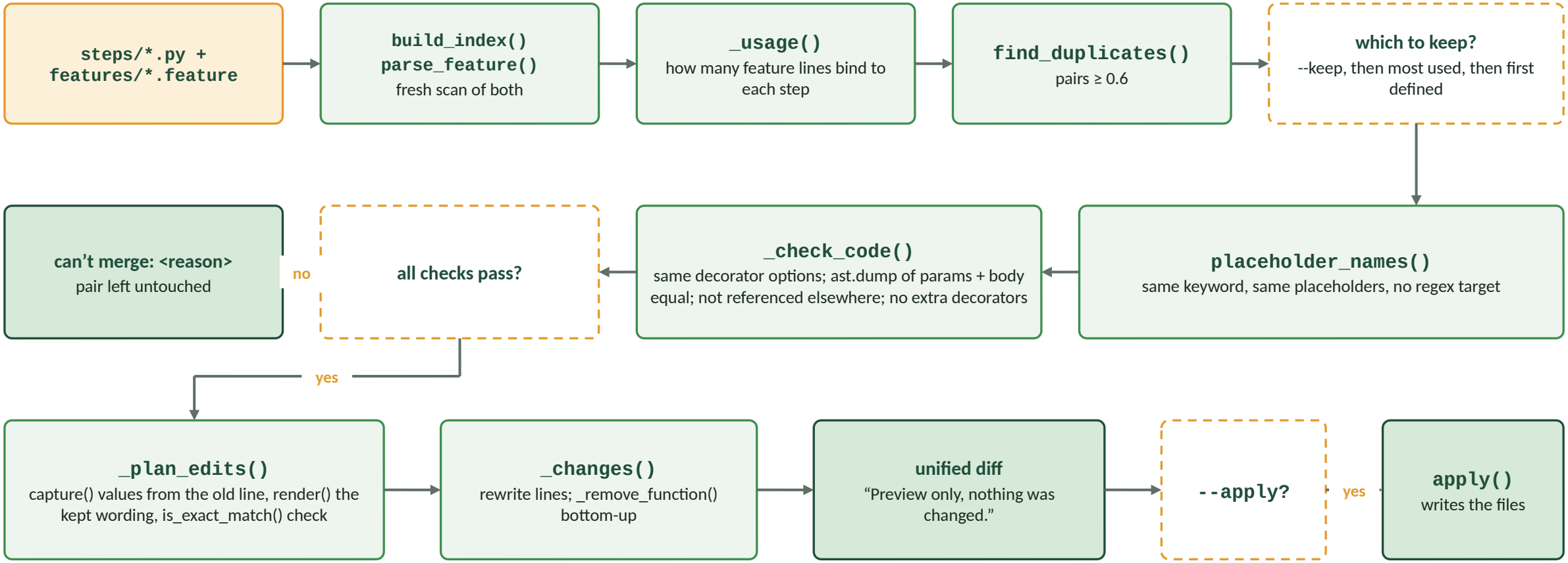
gherkin-copilot dupes (the app's Duplicates tab uses the same function)



Worked example:
"add <arg> cart" vs "put <arg> cart": characters 0.79, words 2 of 4 = 0.50, average 0.64.

merge: plan, refuse anything unsafe, then preview

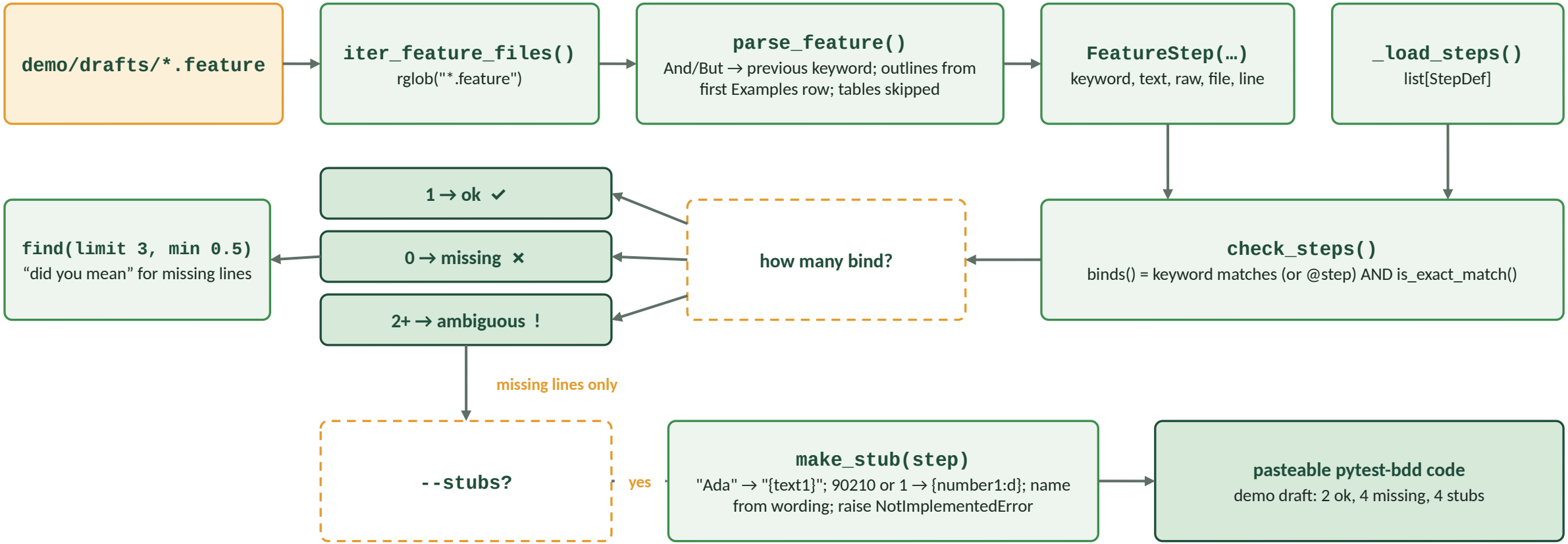
gherkin-copilot merge demo [--apply] (the app's Merge button uses the same planner)



Demo:
2 pairs merged. 1 feature line rewritten ("I put the … in my cart" → "I add … to the cart"), put_in_cart() and user_on_login_page() removed. All 5 tests still pass.

check and --stubs: is every scenario line backed by a step?

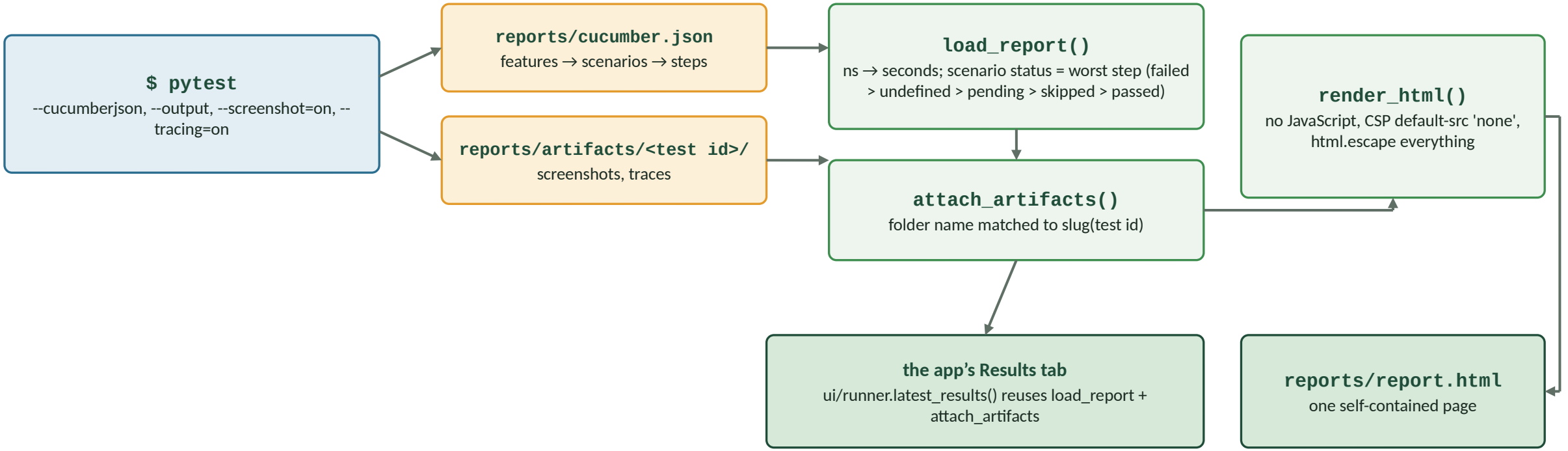
gherkin-copilot check demo/drafts --path demo/steps --stubs



Exit code:
0 only when every line is ok, so CI can fail a build on a missing or ambiguous step.

report: from pytest's results to one HTML page

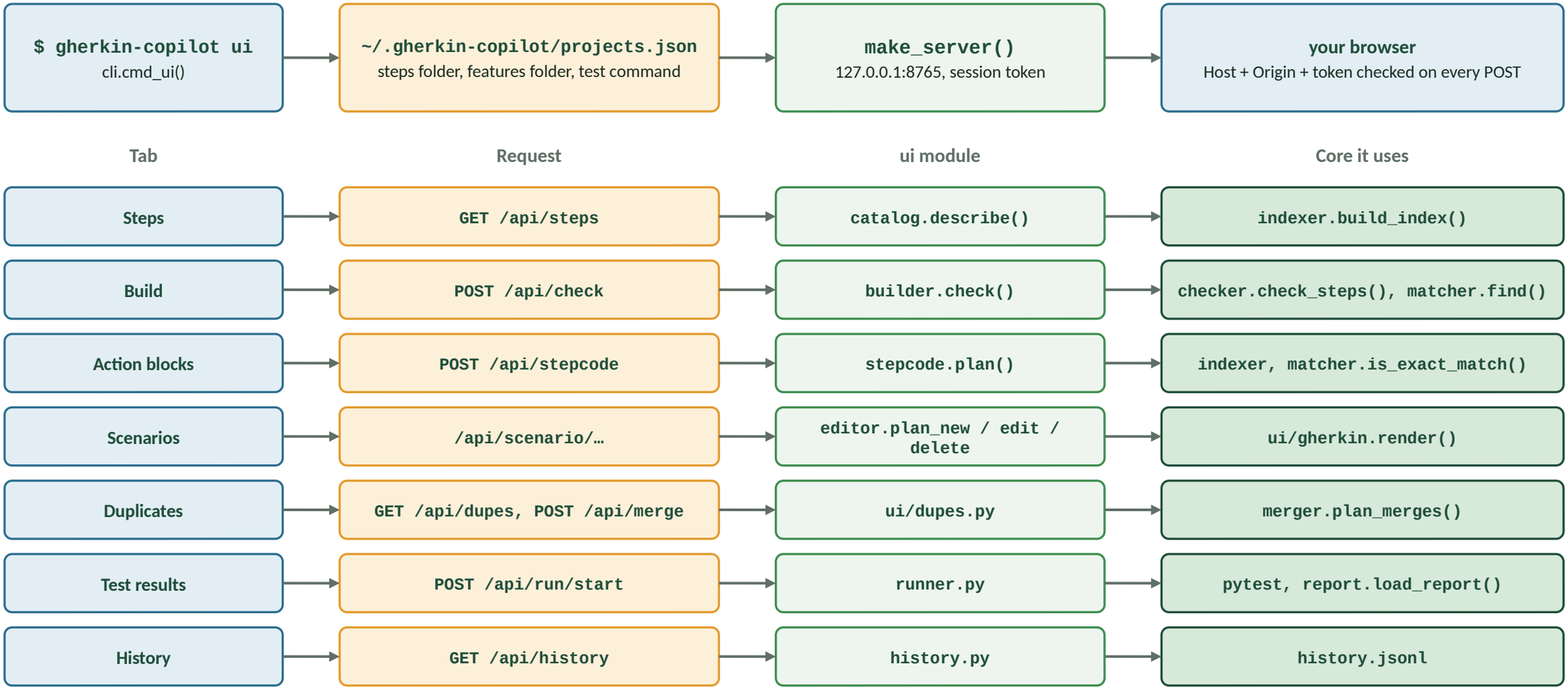
gherkin-copilot report reports/cucumber.json -o reports/report.html



Demo:
5 scenarios, all passing in a real browser in about 4 seconds.

The app: which tab calls which function

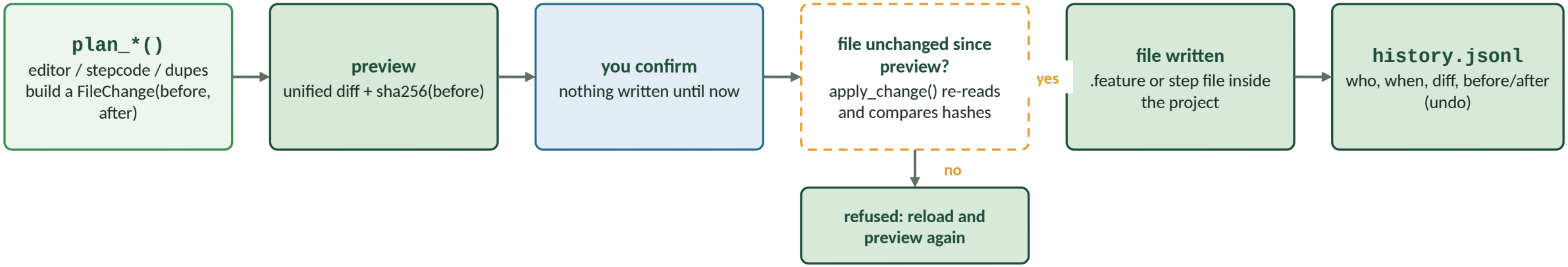
gherkin-copilot ui → http://127.0.0.1:8765



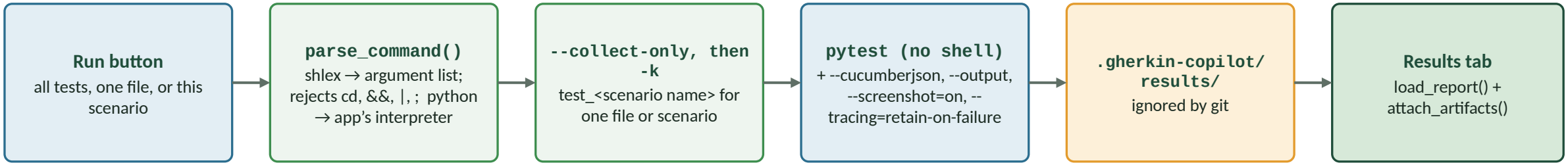
The app: how Save and Run work

Every write goes through the top row. The Run button is the bottom row.

Save



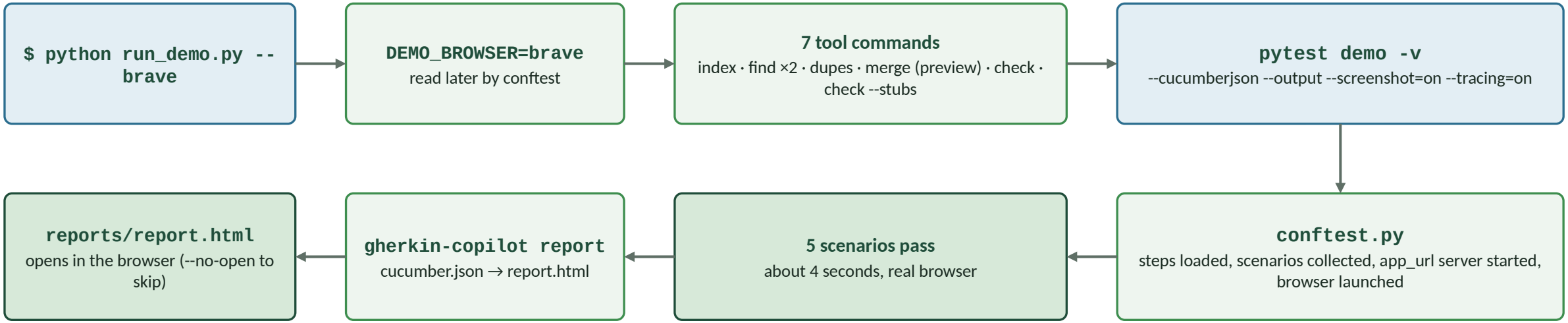
Run



Limits on Run:
only the command saved in the project settings, one run per project at a time, a stop button and a time limit.

run_demo.py: the whole demo, end to end

python run_demo.py --brave



All subprocesses
are started with an argument list (subprocess.run([...])), never through a shell.